

Day 1 - Variables, Types, and Equality

Day 1 — Variables, Types, and Equality

If you haven't read [day00-primer/lesson.md](#) yet, read that first.

Objectives

- Declare variables correctly with `let`, `const`, and understand why `var` is avoided in modern code
- Meet JavaScript's core data types
- Understand `undefined` vs `null` — two different "nothing" values, which trips up every beginner at least once
- Understand the difference between `==` and `===`, and why professional JavaScript code almost always uses `===`
- Understand type coercion — JavaScript's tendency to automatically convert values between types, sometimes surprisingly

Declaring variables: `let`, `const`, and (avoid) `var`

```
let age = 25;           // a variable whose value CAN be reassigned later
const name = "Ana";     // a variable whose value CANNOT be reassigned after this line

age = 26;               // fine -- age was declared with let
name = "Bo";            // TypeError: Assignment to constant variable.
```

`let` declares a variable you intend to change later; `const` declares one you don't. **A strong, widely-followed convention in professional JavaScript: default to `const` for everything, and only use `let` when you specifically know you'll need to reassign the variable later.** This isn't just style — a `const` tells anyone reading your code (including future-you) "this value never changes after this point," which makes code easier to reason about; if you see `let`, you know to keep an eye out for where it gets reassigned.

You'll frequently see a third keyword, `var`, in older JavaScript code and tutorials. **Avoid it in new code.** `var` has looser, more confusing rules about which part of your code can see it (its "scope" — full treatment Day 4) that were considered a design mistake in hindsight; `let` and `const` (introduced later in the language's life, in a version called ES6/ES2015) fix those problems and are what all modern JavaScript uses. You should be able to recognize `var` when you see it in existing code, but never choose to write it yourself.

Important: `const` prevents reassigning the variable name, but does NOT make the value itself unchangeable, if that value is a mutable type (like the objects and arrays you'll meet Day 3):

```
const numbers = [1, 2, 3];
numbers.push(4);      // this is FINE -- we're not reassigning `numbers`, just modifying the array
console.log(numbers); // [1, 2, 3, 4]

numbers = [9, 9, 9];  // TypeError -- THIS would be reassigning `numbers` itself, which is not allowed
```

This distinction — "the name can't be reassigned" versus "the value can't be changed" — will feel much more familiar once you've been through Day 1's mutability discussion in the Python track, if you've done that; if not,

don't worry, Day 3 of this track covers exactly this idea in depth once you've met arrays and objects properly.

The core data types

JavaScript calls its basic data types **primitives**. Check any value's type with the `typeof` operator:

```
typeof 42           // "number"
typeof 3.14         // "number" -- JavaScript has only ONE numeric type, unlike Python's separate
typeof "hello"      // "string"
typeof true         // "boolean"
typeof undefined    // "undefined"
typeof null         // "object" -- a famous, long-standing bug in JavaScript itself, exp
```

Type	Example	Notes
number	42, 3.14, -7	JavaScript has exactly one numeric type for everything — no separate "integer" type like Python's <code>int</code> . Internally it's always a floating-point number, which occasionally causes the same kind of rounding quirk you may have seen elsewhere: <code>0.1 + 0.2</code> is <code>0.30000000000000004</code> , not exactly <code>0.3</code> .
string	"hi", 'hi', `hi`	Text. Single quotes, double quotes, and backticks (template literals — full treatment Day 5) are all valid; be consistent within a project.
boolean	true, false	Yes/no. Lowercase, unlike Python's capitalized <code>True/False</code> — a very common typo for anyone switching between the two languages.
undefined	undefined	JavaScript's way of saying "this variable exists, but nothing has been assigned to it yet." You'll see this constantly, often unintentionally, once you start writing functions (Day 4).

<code>null</code>	<code>null</code>	JavaScript's way of saying "this variable deliberately, explicitly has no value" — a value a programmer chose to assign on purpose, as opposed to <code>undefined</code> , which usually means "nothing has been set yet."
<code>object</code>	<code>{}, []</code>	Everything else — collections of data, full treatment Day 3.

`undefined` vs `null` — two different flavors of "nothing"

This confuses every beginner (and plenty of experienced developers) at least once, so let's be precise: **`undefined` is what JavaScript gives you automatically** when something hasn't been given a value — a variable you declared but never assigned, a function parameter nobody passed a value for (Day 4), a property that doesn't exist on an object (Day 3). **`null` is a value a programmer deliberately assigns** to explicitly say "there is genuinely no value here, on purpose" — you will type `null` yourself; you will rarely if ever type `undefined` yourself.

```
let x;
console.log(x);           // undefined -- declared, but never given a value

let y = null;
console.log(y);           // null -- deliberately set to "nothing" by the programmer
```

A practical rule many teams follow: use `null` in your own code whenever you want to explicitly represent "no value" (e.g., "no user is currently logged in" might be `currentUser = null`), and treat `undefined` as something that shows up on its own, signaling "this wasn't set" rather than something you assign by hand.

`==` versus `===` — and why you should almost always use `===`

JavaScript has **two** different equality operators, and the difference between them is one of the most important things to get right as a beginner.

`===` ("strict equality") checks whether two values are equal **and** the same type — no conversion, no guessing:

```
5 === 5           // true
5 === "5"         // false -- different types (number vs string), so NOT equal
```

`==` ("loose equality") first tries to **convert** the two values to a matching type, and only then compares them — and its conversion rules are notoriously full of surprising special cases:

```
5 == "5"          // true -- "5" gets converted to the number 5 first, then compared
0 == false        // true -- false gets converted to 0
"" == false       // true
null == undefined // true -- these two DIFFERENT "nothing" values are considered == equal
null === undefined // false -- with strict equality, they're correctly treated as different
```

The near-universal professional convention: always use `===` and `!==` (strict "not equal"), never `==` or `!=`. The conversion rules behind `==` are inconsistent enough that even experienced developers can't always predict them without looking them up, and using `===` eliminates an entire category of subtle bugs before they can happen, by simply refusing to compare values of different types at all. You'll see `==` in older code and tutorials — recognize it, but don't write it yourself.

Type coercion — JavaScript's habit of automatically converting between types

Type coercion is what JavaScript does when an operation involves two different types — instead of raising an error the way Python often would, JavaScript tries to automatically convert one or both values so the operation can proceed, guided by its own internal rules. You already saw this above with `==`; it also happens with ordinary operators like `+`:

```
"5" + 3          // "53"  -- the number 3 is converted to the STRING "3", then the two strings are joined
5 + "3"          // "53"  -- same thing, other order
5 + 3            // 8      -- both numbers, ordinary addition, no coercion needed
"5" - 3          // 2      -- surprisingly, MINUS converts "5" to the number 5 first, then subtracts
```

Notice `+` and `-` behave completely differently when a string is involved: `+` treats a string as "please join text together," while `-` (which makes no sense for text) instead tries to convert the string to a number first. This inconsistency is exactly why relying on automatic coercion is considered risky in real code — **when you need a specific type, convert explicitly yourself**, rather than hoping JavaScript's automatic rules happen to do what you want:

```
Number("5")      // 5 -- explicit, deliberate conversion to a number
String(5)         // "5" -- explicit, deliberate conversion to a string
Boolean(0)        // false -- explicit, deliberate conversion to a boolean
```

Truthy and falsy — which values count as "true enough" in a condition

You'll meet `if` properly tomorrow (Day 2), but since it connects directly to today's types: every JavaScript value has an implicit true/false-ish answer when used somewhere expecting a boolean. The **falsy** values — everything that counts as "false-ish" — are: `false`, `0`, `""` (empty string), `null`, `undefined`, and `NaN` (a special "Not a Number" value you'll encounter when a math operation fails, e.g. `Number("abc")`). **Every other value is truthy**, including things that might surprise you at first: an empty array `[]` and an empty object `{}` are BOTH truthy in JavaScript — this is a genuine, important difference from Python, where an empty list/dict is falsy. Keep this specific difference in mind if you're coming from the Python track.

Exercises

Open `exercises.js`, implement each function where you see `// TODO`, and run it with `node exercises.js` to check your work against the printed PASS/FAIL output.